Greedy Algorithms

1. Assign Cookies

You are given two arrays:

- `student[]` where `student[i]` is the minimum cookie size needed by the i-th student
- `cookie[]` where `cookie[j]` is the size of the j-th cookie

Each student can receive **at most one cookie**, and a cookie can be assigned only if `cookie[j] >= student[i]`.

Your task is to **maximize the number of students who receive a cookie**.

Example:
Input: `student = [1,2,3]`, `cookie = [1,1]`
Only one student can be satisfied, so output is 1.

Input: `student = [1,2]`, `cookie = [1,2,3]`
Both students can be satisfied, so output is 2.

Approach 1: Memoization (Top-Down DP)

Algorithm

We try all possible ways of assigning cookies to students using recursion.

Steps:

1. Sort both `student` and `cookie` arrays.
2. Use recursion with two indices:
    - `i` for current student
    - `j` for current cookie
3. If the cookie cannot satisfy the student, skip the cookie.
4. If it can satisfy:

- Option 1: Assign the cookie and move to next student and cookie.
- Option 2: Skip the cookie and try the next one.

5. Use a 2D memo table to store already computed results.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findContentChildren(vector<int>& student, vector<int>& cookie)
{
        sort(student.begin(), student.end());
        sort(cookie.begin(), cookie.end());

        vector<vector<int>> memo(student.size(),
vector<int>(cookie.size(), -1));
        return helper(0, 0, student, cookie, memo);
    }

private:
    int helper(int i, int j, vector<int>& student, vector<int>&
cookie,
               vector<vector<int>>& memo) {

        if(i >= student.size() || j >= cookie.size())
            return 0;

        if(memo[i][j] != -1)
            return memo[i][j];

        int result = 0;

        if(cookie[j] >= student[i]) {
            result = max(result, 1 + helper(i + 1, j + 1, student,
cookie, memo));
        }
```

```
        result = max(result, helper(i, j + 1, student, cookie, memo));
        return memo[i][j] = result;
    }
};

int main() {
    vector<int> student = {1, 2, 3};
    vector<int> cookie = {1, 1};

    Solution sol;
    cout << sol.findContentChildren(student, cookie);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(n * m)`
- Space Complexity: `O(n * m)` (memo table + recursion stack)

Approach 2: Tabulation (Bottom-Up DP)

Algorithm

We build a DP table where `dp[i][j]` represents the maximum number of students satisfied starting from `student[i]` and `cookie[j]`.

Steps:

1. Sort both arrays.
2. Create a `(n+1) x (m+1)` DP table initialized with 0.
3. Fill the table from bottom-right to top-left.
4. At each position:

- Option 1: Skip current cookie.
- Option 2: Assign cookie if possible.

5. Final answer is `dp[0][0]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findContentChildren(vector<int>& student, vector<int>& cookie)
{
        int n = student.size();
        int m = cookie.size();

        sort(student.begin(), student.end());
        sort(cookie.begin(), cookie.end());

        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

        for(int i = n - 1; i >= 0; i--) {
            for(int j = m - 1; j >= 0; j--) {
                int skip = dp[i][j + 1];
                int take = 0;
                if(cookie[j] >= student[i])
                    take = 1 + dp[i + 1][j + 1];

                dp[i][j] = max(skip, take);
            }
        }
        return dp[0][0];
    }
};

int main() {
    vector<int> student = {1, 2};
```

```
    vector<int> cookie = {1, 2, 3};

    Solution sol;
    cout << sol.findContentChildren(student, cookie);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(n * m)`
- Space Complexity: `O(n * m)`

Approach 3: Optimal Greedy Approach (Two Pointers)

Algorithm

To maximize satisfied students:

- Give the **smallest sufficient cookie** to the **least greedy student**.

Steps:

1. Sort both `student` and `cookie` arrays.
2. Use two pointers:
    - One for students
    - One for cookies
3. If current cookie satisfies current student:
    - Assign it and move both pointers.
4. Otherwise:
    - Move to the next cookie.

5. Count how many students are satisfied.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findContentChildren(vector<int>& student, vector<int>& cookie)
{
        sort(student.begin(), student.end());
        sort(cookie.begin(), cookie.end());

        int i = 0, j = 0;

        while(i < student.size() && j < cookie.size()) {
            if(cookie[j] >= student[i]) {
                i++;
            }
            j++;
        }
        return i;
    }
};

int main() {
    vector<int> student = {1, 2, 3};
    vector<int> cookie = {1, 1};

    Solution sol;
    cout << sol.findContentChildren(student, cookie);
    return 0;
}
```

Complexity Analysis

- Time Complexity: $O(n \log n + m \log m)$ due to sorting

- Space Complexity: $O(1)$

2. Fractional Knapsack Problem : Greedy Approach

You are given N items, where each item has a value and a weight. You are also given a knapsack with maximum capacity W.
Your goal is to **maximize the total value** inside the knapsack.

You are allowed to:

- Take an item completely, or
- Take **any fraction** of an item.

This makes the problem different from the 0/1 Knapsack problem.

Example:
If an item has value 120 and weight 30, then taking 20 weight from it gives value 80.

Approach: Greedy (Value-to-Weight Ratio)

Algorithm

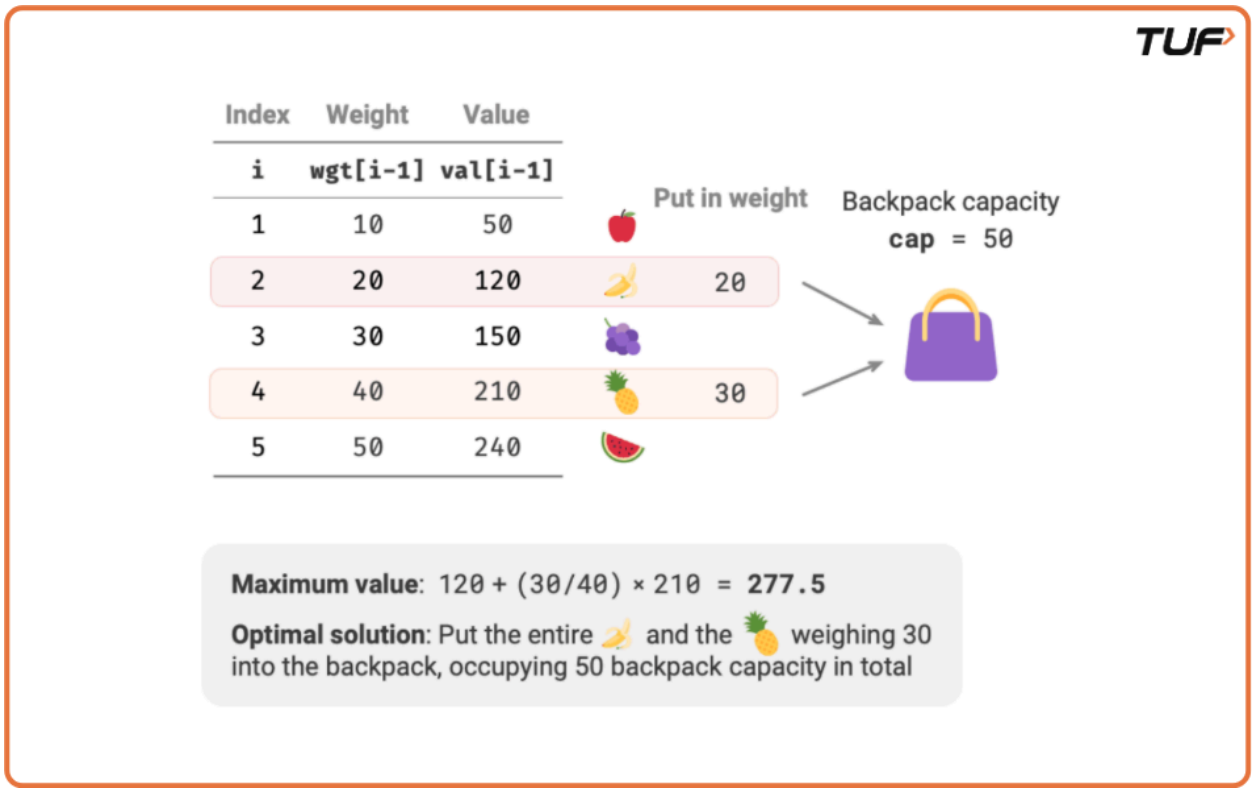
Since items can be broken into smaller parts, the best strategy is to **always take the item with the highest value per unit weight first**.

Steps:

1. For each item, calculate its value-to-weight ratio.
2. Sort all items in **descending order** of value/weight ratio.
3. Initialize current knapsack weight = 0 and total value = 0.

4. Traverse the sorted items:
    - If the entire item fits in the remaining capacity, take it fully.
    - Otherwise, take only the fraction that fits and add proportional value.
5. Stop once the knapsack is full.

This greedy choice works because taking the highest ratio first always gives maximum value contribution per unit weight.



Code

```
#include <bits/stdc++.h>
using namespace std;

// Structure to represent an item
struct Item {
    int value;
    int weight;
```

```
};

class Solution {
public:
    // Jis item ka value/weight ratio zyada ho, usse pehle uthao
    static bool comp(Item a, Item b) {
        double r1 = (double)a.value / a.weight;
        double r2 = (double)b.value / b.weight;
        return r1 > r2;
    }
/comp(a, b) == true 👉 Matlab: a ko b se pehle aana chahiye

    // Function to get maximum value in fractional knapsack
    double fractionalKnapsack(int W, Item arr[], int n) {

        // Sort items by decreasing value/weight ratio
        sort(arr, arr + n, comp);

        int curWeight = 0;
        double finalValue = 0.0;

        for (int i = 0; i < n; i++) {

            // If full item can be taken
            if (curWeight + arr[i].weight <= W) {
                curWeight += arr[i].weight;
                finalValue += arr[i].value;
            }
            // Take fractional part
            else {
                int remain = W - curWeight;
                finalValue += (arr[i].value / (double)arr[i].weight) *
remain;
                break;
            }
        }
        return finalValue;
    }
```

```
};

int main() {
    int n = 3, W = 50;
    Item arr[n] = {{100,20}, {60,10}, {120,30}};

    Solution obj;
    double ans = obj.fractionalKnapsack(W, arr, n);

    cout << fixed << setprecision(6) << ans;
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(n log n)`
  Sorting items by value-to-weight ratio takes `O(n log n)` and iterating once takes `O(n)`.

- **Space Complexity:** `O(1)`
  No extra data structures are used apart from variables.

3. Find Minimum Number of Coins

You are given a value `V` (in rupees). You have an **infinite supply** of Indian currency denominations:
`{1, 2, 5, 10, 20, 50, 100, 500, 1000}`.

Your task is to find the **minimum number of coins/notes** required to make the value `V`.

Example:
Input: `V = 70`
We can use `50 + 20`, so output is 2.

Input: `V = 121`
We can use `100 + 20 + 1`, so output is `3`.

Approach: Greedy Approach

Algorithm

This problem works perfectly with a greedy strategy because Indian currency denominations are designed to be greedy-friendly.

Steps:

1. Store all coin denominations in an array in increasing order.
2. Start from the **largest denomination**.
3. While the current coin value is less than or equal to `V`:
    - Subtract the coin value from `V`.
    - Store the coin in the answer list.
4. Move to the next smaller denomination.
5. Continue until `V` becomes `0`.

This ensures we always use the largest possible coin first, minimizing the total count.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Function to compute minimum coins required for value V
vector<int> minCoins(int V) {
    int coins[] = {1, 2, 5, 10, 20, 50, 100, 500, 1000};
    int n = 9;

    vector<int> ans;
```

```
    // Traverse from largest to smallest denomination
    for (int i = n - 1; i >= 0; i--) {
        while (V >= coins[i]) {
            V -= coins[i];
            ans.push_back(coins[i]);
        }
    }
    return ans;
}

int main() {
    int V = 121;

    vector<int> result = minCoins(V);

    cout << "Minimum number of coins: " << result.size() << endl;
    cout << "Coins used: ";
    for (int coin : result) {
        cout << coin << " ";
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(V)`
  In the worst case, we may subtract 1 repeatedly (all 1-value coins).

- **Space Complexity:** `O(V)`
  In the worst case, the result array may store `V` coins of value 1.

4. Lemonade Change

You are given an array `bills` where each element represents the bill a customer pays with.
Each lemonade costs **5$**. Customers can only pay using **5$, 10$, or 20$** bills.
Initially, you have **no change**.

Your task is to determine whether you can provide the **correct change to every customer in order**.
Return `true` if possible, otherwise return `false`.

Example:
Input: `bills = [5, 5, 5, 10, 20]`
Output: `true`
Because correct change can be given to every customer.

Input: `bills = [5, 5, 10, 10, 20]`
Output: `false`
Because at the last customer, it is not possible to give 15$ change.

Approach 1: Greedy Counting

Algorithm

We simulate the process of giving change to each customer using a greedy strategy.

Steps:

1. Maintain two counters:
    - `five` → number of 5$ bills
    - `ten` → number of 10$ bills
2. Traverse the `bills` array one by one:
    - If the bill is 5:
        - No change needed, increment `five`.

- If the bill is `10`:
    - We must give one 5$ as change.
    - If `five > 0`, decrement `five` and increment `ten`.
    - Otherwise, return `false`.
- If the bill is `20`:
    - We need to give 15$ change.
    - Prefer giving one 10$ and one 5$ (best option).
    - If not possible, try giving three 5$ bills.
    - If neither is possible, return `false`.

3. If all customers are processed successfully, return `true`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool lemonadeChange(vector<int>& bills) {
        int five = 0;
        int ten = 0;

        for (int bill : bills) {
            if (bill == 5) {
                five++;
            }
            else if (bill == 10) {
                if (five == 0) return false;
                five--;
```

```
                ten++;
            }
            else { // bill == 20
                if (ten > 0 && five > 0) {
                    ten--;
                    five--;
                }
                else if (five >= 3) {
                    five -= 3;
                }
                else {
                    return false;
                }
            }
        }
        return true;
    }
};

int main() {
    vector<int> bills = {5, 5, 5, 10, 20};
    Solution obj;
    cout << obj.lemonadeChange(bills);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each customer is processed exactly once.

- **Space Complexity:** `O(1)`
  Only constant extra variables (`five`, `ten`) are used.

5. Valid Parenthesis Checker

You are given a string s that contains only three characters:
`'('`, `')'`, and `'*'`.

The string is considered **valid** if:

- Every `'('` has a corresponding `')'`.
- Every `')'` has a corresponding `'('`.
- `'('` always comes before its matching `')'`.
- `'*'` can act as:
    - `'('`
    - `')'`
    - an empty string `""`

You need to determine whether the string can be interpreted as a valid parenthesis string.

Example:
Input: `s = (*))`
Here * can be replaced with `'('`, making the string `(())`, which is valid.

Input: `s = *(()`
No replacement of * can make the string valid, so the result is false.

Approach 1: Brute Force (Recursion / Backtracking)

Algorithm

We try **all possible interpretations** of `'*'`.

Steps:

1. Use a recursive function with:
    - current index `i`
    - count of currently open brackets `open`
2. If `open < 0`, return false (too many closing brackets).
3. If we reach the end of the string:
    - return true if `open == 0`
    - otherwise return false
4. If current character is:
    - `'('` → increase `open`
    - `')'` → decrease `open`
    - `'*'` → try all three cases:
        - treat as empty
        - treat as `'('`
        - treat as `')'`
5. If any recursive path returns true, the string is valid.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isValid(string& s, int i, int open) {
        if (open < 0) return false;

        if (i == s.size())
```

```
            return open == 0;

        if (s[i] == '(') {
            return isValid(s, i + 1, open + 1);
        }
        else if (s[i] == ')') {
            return isValid(s, i + 1, open - 1);
        }
        else { // '*'
            return isValid(s, i + 1, open) ||
                   isValid(s, i + 1, open + 1) ||
                   isValid(s, i + 1, open - 1);
        }
    }
};

int main() {
    string s;
    cin >> s;

    Solution sol;
    if (sol.isValid(s, 0, 0))
        cout << "True";
    else
        cout << "False";

    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(3^n)` in the worst case
  Every `'*'` creates three recursive branches.

- Space Complexity: `O(n)`
  Due to recursion stack depth.

Approach 2: Optimal Greedy Approach

Algorithm

Instead of checking all combinations, we **track a range of possible open brackets**.

Maintain:

- `minOpen`: minimum possible open brackets
- `maxOpen`: maximum possible open brackets

Steps:

1. Traverse the string from left to right.
2. For each character:
    - `'('` → `minOpen++`, `maxOpen++`
    - `')'` → `minOpen--`, `maxOpen--`
    - `'*'` →
        - `minOpen--` (treat as `')'`)
        - `maxOpen++` (treat as `'('`)
3. If `maxOpen < 0`, return false.
4. Ensure `minOpen` never goes below 0.
5. At the end, if `minOpen == 0`, return true.

`minOpen`

👉 Is point tak **minimum kitne '(' open ho sakte hain**
(optimistically * ko ') ' ya empty maan ke)

`maxOpen`

👉 Is point tak **maximum kitne '(' open ho sakte hain**
(optimistically * ko ' (' maan ke)

This works because we only care whether **some valid interpretation exists**.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isValid(string s) {
        int minOpen = 0, maxOpen = 0;

        for (char c : s) {
            if (c == '(') {
                minOpen++;
                maxOpen++;
            }
            else if (c == ')') {
                minOpen--;
                maxOpen--;
            }
            else { // '*'
                minOpen--;
                maxOpen++;
            }

            if (maxOpen < 0) return false;
            minOpen = max(minOpen, 0);
        }

        return minOpen == 0;
    }
};

int main() {
    string s;
    cin >> s;
```

```
    Solution sol;
    if (sol.isValid(s))
        cout << "True";
    else
        cout << "False";

    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(n)`
  Single pass through the string.

- Space Complexity: `O(1)`
  Only constant extra variables are used.

6. N Meetings in One Room

You are given two arrays `start[]` and `end[]` of size `N`.
For each meeting `i`:

- `start[i]` is the starting time
- `end[i]` is the ending time

Only **one meeting** can take place in the room at any time.
Your task is to find the **maximum number of meetings** that can be conducted and **print the order (1-based index)** in which these meetings will be performed.

Example:
Input:
`start = {1,3,0,5,8,5}`
`end = {2,4,5,7,9,9}`

Output:
`[1, 2, 4, 5]`

These meetings do not overlap and allow the maximum count.

Approach: Greedy (Sort by Ending Time)

Algorithm

To schedule the maximum number of meetings, we always prefer the meeting that **ends earliest**, because it frees the room as soon as possible for the next meetings.

Steps:

1. Store each meeting as a tuple of `(end time, start time, meeting index)`.
2. Sort all meetings by their **end time**.
3. Select the first meeting and set its end time as the last occupied time.
4. Traverse the remaining meetings:
    - If a meeting's start time is **strictly greater** than the last selected meeting's end time, select it.
    - Update the last occupied end time.
5. Continue until all meetings are checked.
6. Return the selected meeting indices in order.

This greedy choice guarantees the maximum number of non-overlapping meetings.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
```

```
    vector<int> maxMeetings(vector<int>& start, vector<int>& end) {
        vector<tuple<int,int,int>> meetings;

        for(int i = 0; i < start.size(); i++) {
            meetings.push_back({end[i], start[i], i + 1});
        }

        sort(meetings.begin(), meetings.end());

        vector<int> result;
        int lastEnd = -1;

        for(auto &m : meetings) {
            int e = get<0>(m);
            int s = get<1>(m);
            int idx = get<2>(m);

            if(s > lastEnd) {
                result.push_back(idx);
                lastEnd = e;
            }
        }
        return result;
    }
};

int main() {
    vector<int> start = {1,3,0,5,8,5};
    vector<int> end   = {2,4,5,7,9,9};

    Solution sol;
    vector<int> res = sol.maxMeetings(start, end);

    for(int x : res) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N log N)`
  Sorting meetings by end time dominates the complexity.

- **Space Complexity:** `O(N)`
  Extra space is used to store meeting tuples and the result list.

7. Jump Game – I

You are given an array `nums` where `nums[i]` represents the **maximum jump length** from index `i`.
Starting from index `0`, determine whether you can reach the **last index**.

Intuition (Greedy)

Instead of trying all jump possibilities, track the **farthest index** you can reach at any point.

- If at any index `i`, `i` is **greater than the farthest reachable index**, then this index is unreachable → return `false`.

- Otherwise, update the farthest reachable index using the jump from `i`.

If you finish scanning the array without getting stuck, the last index is reachable.

Algorithm

1. Initialize `maxIndex = 0` (farthest reachable index so far).

2. Traverse the array from left to right.

3. For each index `i`:

    - If `i > maxIndex`, return `false` (we are stuck).

- Update
  `maxIndex = max(maxIndex, i + nums[i])`

4. If the loop ends, return `true`.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool canJump(vector<int>& nums) {
        int maxIndex = 0;

        for (int i = 0; i < nums.size(); i++) {
            // If current index is not reachable
            if (i > maxIndex) return false;

            // Update farthest reachable index
            maxIndex = max(maxIndex, i + nums[i]);
        }
        return true;
    }
};

int main() {
    vector<int> nums = {2, 3, 1, 0, 4};
    Solution sol;

    cout << (sol.canJump(nums) ? "True" : "False") << endl;
    return 0;
}
```

Dry Run (Example)

`nums = [3, 2, 1, 0, 4]`

Index (i)	nums[i]	maxIndex
0	3	3
1	2	3
2	1	3
3	0	3
4	4	❌ unreachable

Since index `4 > maxIndex`, return `false`.

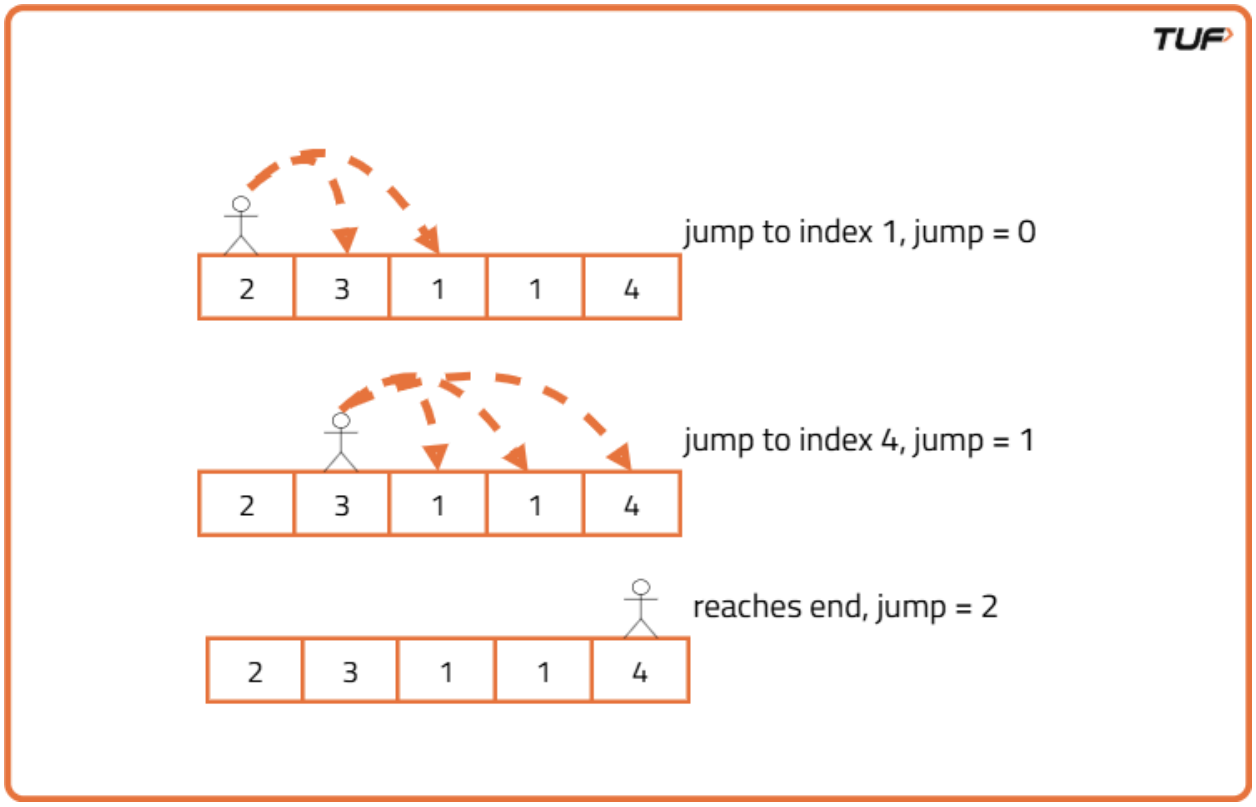
Complexity

- **Time Complexity:** `O(N)`
- **Space Complexity:** `O(1)`

8. Jump Game 2

You are given a 0-indexed array `nums`. Each element `nums[i]` tells the maximum number of steps you can jump forward from index `i`.
You start from index `0` and want to reach the last index `n-1` using the minimum number of jumps.
It is guaranteed that the last index is always reachable.

Example:
`nums = [2,3,1,1,4]`
From index `0`, you can jump to index 1, and from index 1 you can jump directly to index 4.
Minimum jumps = 2.



Approach 1: Brute Force (Recursion)

Algorithm

We try all possible jumps from every index and find the minimum jumps needed to reach the end.

From the current index, we recursively try jumping 1 step, 2 steps, up to `nums[i]` steps.

We return the minimum jumps among all possible paths.

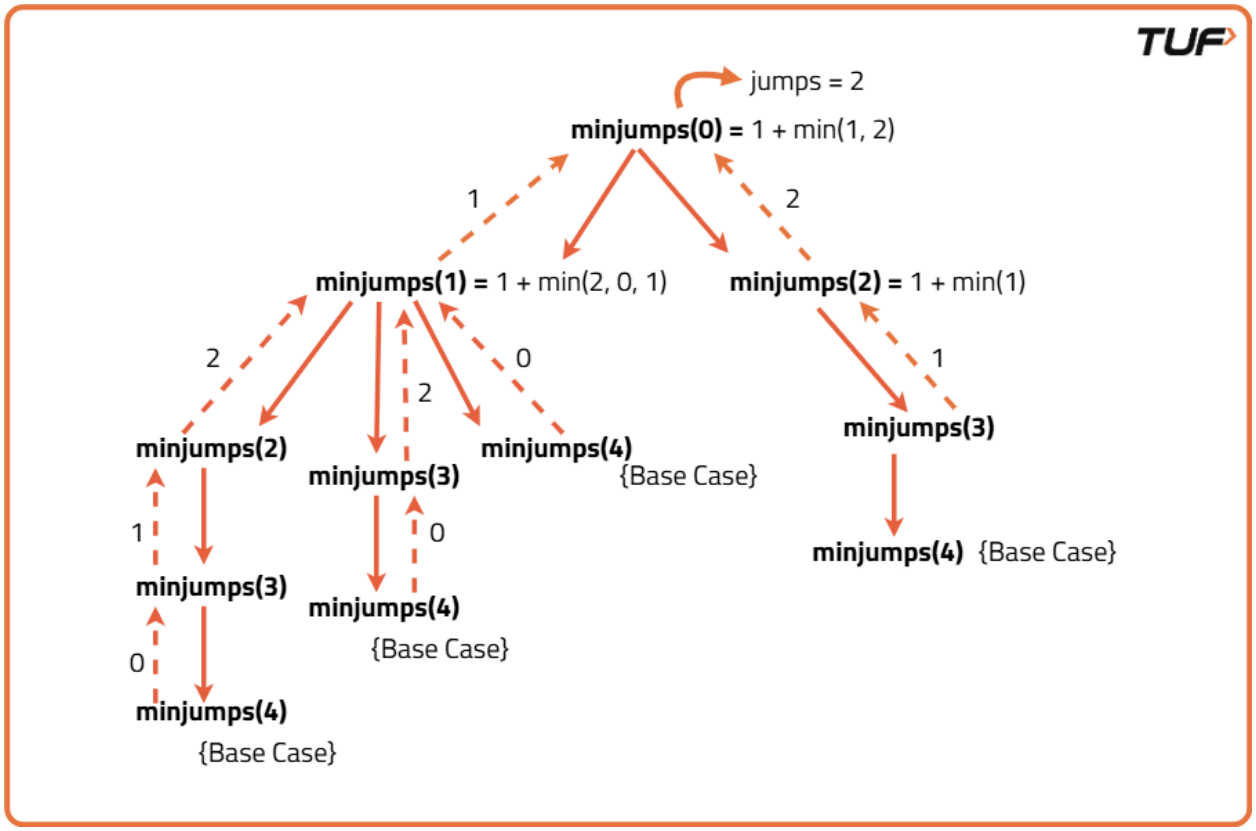
Steps:

- Create a recursive function that takes the current index.
- If the current index is at or beyond the last index, return `0`.
- If `nums[i] == 0`, return `INT_MAX` because no jump is possible.
- Try all jumps from 1 to `nums[i]`.
- For each jump, recursively calculate jumps from the new index.

- Return the minimum jumps found.

Example dry run (nums = [2,3,1,1,4]):

- From index `0`, try jumps to index 1 and 2.
- From index 1, try jumps to index 2, 3, 4.
- Reaching index 4 returns `0`, so total jumps are counted back.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int jump(vector<int>& nums) {
        return minJumps(nums, 0);
```

```
    }

private:
    int minJumps(vector<int>& nums, int position) {
        if (position >= nums.size() - 1) return 0;
        if (nums[position] == 0) return INT_MAX;

        int minStep = INT_MAX;
        for (int j = 1; j <= nums[position]; j++) {
            int res = minJumps(nums, position + j);
            if (res != INT_MAX)
                minStep = min(minStep, 1 + res);
        }
        return minStep;
    }
};

int main() {
    vector<int> nums = {2,3,1,1,4};
    Solution sol;
    cout << sol.jump(nums);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(2^N)`
  Every index explores multiple recursive paths, causing exponential calls.

- Space Complexity: `O(N)`
  Due to recursion stack depth in the worst case.

Approach 2: Better Approach (Dynamic Programming)

Algorithm

We use a DP array where `dp[i]` stores the minimum jumps needed to reach index `i`.
We start from index `0` and update all reachable indices from each position.

Steps:

- Create a dp array of size n and initialize all values to `INT_MAX`.
- Set `dp[0] = 0`.
- For each index `i`, try all jumps from 1 to `nums[i]`.
- Update `dp[i + j] = min(dp[i + j], dp[i] + 1)`.
- The answer is `dp[n-1]`.

Example dry run (nums = [2,3,1,1,4]):

- `dp[0] = 0`
- From index `0`, update `dp[1] = 1`, `dp[2] = 1`
- From index 1, update `dp[4] = 2`
- Final answer = 2

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int jump(vector<int>& nums) {
        int n = nums.size();
        vector<int> dp(n, INT_MAX);
        dp[0] = 0;

        for (int i = 0; i < n; i++) {
            for (int j = 1; j <= nums[i] && i + j < n; j++) {
                dp[i + j] = min(dp[i + j], dp[i] + 1);
            }
        }
```

```
        return dp[n - 1];
    }
};

int main() {
    vector<int> nums = {2,3,1,1,4};
    Solution sol;
    cout << sol.jump(nums);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N^2)`
  Two nested loops where each index may explore up to `N` jumps.

- Space Complexity: `O(N)`
  Extra DP array is used to store minimum jumps.

Approach 3: Optimal Approach (Greedy)

Algorithm
We use a greedy method to always jump as far as possible within the current range.
Each jump expands the reachable range, and we count jumps when the current range ends.

Steps:

- Initialize `jumps = 0`, `currentEnd = 0`, `farthest = 0`.
- Traverse the array until the second last index.
- Update `farthest = max(farthest, i + nums[i])`.
- When `i == currentEnd`, increment jumps and update `currentEnd = farthest`.
- Return `jumps`.

Example dry run (nums = [2,3,1,1,4]):

- At index 0, farthest = 2, currentEnd reached → jumps = 1, currentEnd = 2
- At index 1, farthest = 4
- At index 2, currentEnd reached → jumps = 2, currentEnd = 4
- End reached with 2 jumps.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int jump(vector<int>& nums) {
        int jumps = 0, currentEnd = 0, farthest = 0;

        for (int i = 0; i < nums.size() - 1; i++) {
            farthest = max(farthest, i + nums[i]);
            if (i == currentEnd) {
                jumps++;
                currentEnd = farthest;
            }
        }
        return jumps;
    }
};

int main() {
    vector<int> nums = {2,3,1,1,4};
    Solution sol;
    cout << sol.jump(nums);
    return 0;
}
```

Complexity Analysis

- Time Complexity: $O(N)$
  The array is traversed only once.

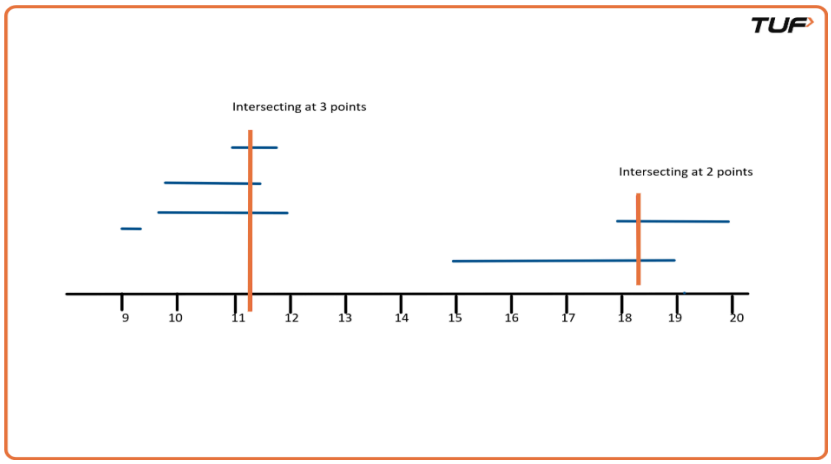
- Space Complexity: $O(1)$
  Only constant extra variables are used.

9. Minimum number of platforms required for a railway

We are given two arrays: one for arrival times and one for departure times of trains at a railway station. Each train needs one platform during the time it is at the station.
The task is to find the minimum number of platforms required so that no train has to wait.

Example:
arr = {9:00, 9:45, 9:55, 11:00, 15:00, 18:00}
dep = {9:20, 12:00, 11:30, 11:50, 19:00, 20:00}

At time 11:00, three trains are present at the station at the same time, so the answer is 3.



Approach 1: Brute Force

Algorithm
We check overlaps between every pair of trains.

For each train, we count how many other trains overlap with it.
The maximum overlap count at any time is the number of platforms needed.

Steps:

- Initialize answer as 1.
- For each train `i`, start a count as 1.
- Compare train `i` with every other train `j`.
- If their time intervals overlap, increase the count.
- Update the maximum count after checking all overlaps for `i`.
- Return the maximum count.

Overlap condition:
Two trains overlap if the arrival of one lies between the arrival and departure of the other.

Example dry run (first example):

- For train at 9:45, it overlaps with trains at 9:55 and 11:00.
- Count becomes 3, which is the maximum.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int countPlatforms(int n, int arr[], int dep[]) {
        int ans = 1;
        for (int i = 0; i < n; i++) {
            int count = 1;
            for (int j = i + 1; j < n; j++) {
                if ((arr[i] >= arr[j] && arr[i] <= dep[j]) ||
                    (arr[j] >= arr[i] && arr[j] <= dep[i])) {
                    count++;
```

```
                }
            }
            ans = max(ans, count);
        }
        return ans;
    }
};

int main() {
    int arr[] = {900,945,955,1100,1500,1800};
    int dep[] = {920,1200,1130,1150,1900,2000};
    int n = sizeof(arr)/sizeof(arr[0]);
    Solution obj;
    cout << obj.countPlatforms(n, arr, dep);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N²)

Because for every train, we compare it with all other trains.

Space Complexity: O(1)

Only constant extra variables are used.

Approach 2: Optimal Approach

Algorithm

We want to know the maximum number of trains present at the station at the same time.

By sorting arrival and departure times separately, we can simulate train movements.

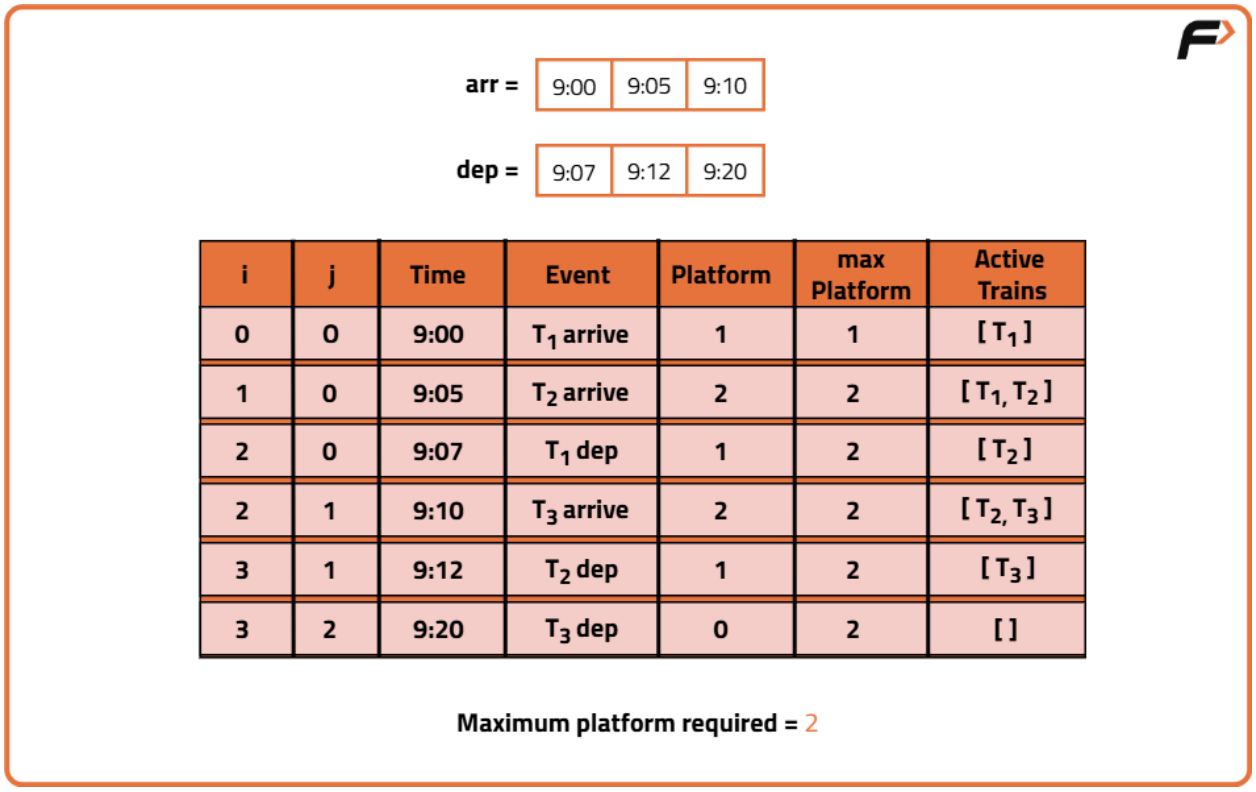
Steps:

- Sort the arrival array and departure array.
- Use two pointers: one for arrivals and one for departures.
- Start with one platform.

- If the next train arrives before or at the same time as the earliest departure, increase platforms.
- If a train departs before the next arrival, decrease platforms.
- Track the maximum number of platforms used at any time.

Example dry run (first example):

- At 9:00 → platforms = 1
- At 9:45 → platforms = 2
- At 9:55 → platforms = 3
- Maximum platforms needed = 3



arr = | 9:00 | 9:05 | 9:10 |

dep = | 9:07 | 9:12 | 9:20 |

i	j	Time	Event	Platform	max Platform	Active Trains
0	0	9:00	T_1 arrive	1	1	[T_1]
1	0	9:05	T_2 arrive	2	2	[T_1, T_2]
2	0	9:07	T_1 dep	1	2	[T_2]
2	1	9:10	T_3 arrive	2	2	[T_2, T_3]
3	1	9:12	T_2 dep	1	2	[T_3]
3	2	9:20	T_3 dep	0	2	[ ]

Maximum platform required = 2

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    int countPlatforms(int n, int arr[], int dep[]) {
        sort(arr, arr + n);
        sort(dep, dep + n);

        int platforms = 1;
        int result = 1;
        int i = 1, j = 0;

        while (i < n && j < n) {
            if (arr[i] <= dep[j]) {
                platforms++;
                i++;
            } else {
                platforms--;
                j++;
            }
            result = max(result, platforms);
        }
        return result;
    }
};

int main() {
    int arr[] = {900,945,955,1100,1500,1800};
    int dep[] = {920,1200,1130,1150,1900,2000};
    int n = sizeof(arr)/sizeof(arr[0]);
    Solution obj;
    cout << obj.countPlatforms(n, arr, dep);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N log N)

Sorting both arrival and departure arrays takes $N \log N$ time.

Space Complexity: O(1)
No extra space is used apart from variables.

10. Job Sequencing Problem

You are given N jobs. Each job has an id, a deadline, and a profit.
Each job takes exactly one unit of time, and only one job can be done at a time.
A job gives profit only if it is completed on or before its deadline.
Your task is to find:

1. The maximum number of jobs that can be done
2. The maximum profit that can be earned

Example:
Jobs = {(1,4,20), (2,1,10), (3,1,40), (4,1,30)}

If we select the job with profit 40 first (deadline 1) and then the job with profit 20 (deadline 4), we can do 2 jobs with total profit 60.

Approach:

Algorithm
We always try to take the job with the highest profit first.
For each job, we try to schedule it as late as possible before its deadline so that earlier time slots remain free for other jobs.

Steps:

- Sort all jobs in decreasing order of profit.
- Find the maximum deadline among all jobs.
- Create an array `slot[]` of size `maxDeadline + 1` and initialize all values to `-1` (free slot).

- Traverse jobs one by one (highest profit first).
- For the current job, check from its deadline backwards to find a free slot.
- If a free slot is found, assign the job, increase job count, and add profit.
- Continue for all jobs.
- Return total jobs done and total profit.

Example dry run (Example 1):
Sorted by profit → (3,1,40), (4,1,30), (1,4,20), (2,1,10)

- Job 3 goes to slot 1 → profit = 40
- Job 4 cannot be placed (slot 1 already used)
- Job 1 goes to slot 4 → profit = 60
- Job 2 cannot be placed
  Result: 2 jobs, profit = 60

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Job {
    int id;
    int dead;
    int profit;
};

class Solution {
public:
    static bool comparison(Job a, Job b) {
        return a.profit > b.profit;
    }

    pair<int,int> JobScheduling(Job arr[], int n) {
```

```
        sort(arr, arr + n, comparison);

        int maxi = arr[0].dead;
        for (int i = 1; i < n; i++) {
            maxi = max(maxi, arr[i].dead);
        }

        int slot[maxi + 1];
        for (int i = 0; i <= maxi; i++)
            slot[i] = -1;

        int countJobs = 0, jobProfit = 0;

        for (int i = 0; i < n; i++) {
            for (int j = arr[i].dead; j > 0; j--) {
                if (slot[j] == -1) {
                    slot[j] = i;
                    countJobs++;
                    jobProfit += arr[i].profit;
                    break;
                }
            }
        }
        return make_pair(countJobs, jobProfit);
    }
};

int main() {
    int n = 4;
    Job arr[n] = {{1,4,20},{2,1,10},{3,2,40},{4,2,30}};
    Solution ob;
    pair<int,int> ans = ob.JobScheduling(arr, n);
    cout << ans.first << " " << ans.second;
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N log N) + O(N * M)

- Sorting jobs takes `O(N log N)`
- For each job, we may scan up to `M` slots (maximum deadline)

Space Complexity: O(M)

- Slot array of size equal to maximum deadline is used

11. Candy

A line of `N` kids is standing in a row. Each kid has a rating given in the array `ratings`.
Candies must be distributed using the following rules:

- Every child must get at least one candy.
- A child with a higher rating than an adjacent child must get more candies than that neighbor.

Return the minimum number of candies required.

Example:
ratings = [1, 0, 5]
Candies distribution = [2, 1, 2]
Total candies = 5

Approach 1: Brute Force

Algorithm
We start by giving every child one candy. Then we repeatedly fix violations of the rules by scanning from left to right and from right to left until no changes are needed.

Steps:

- Initialize a candies array with all values as 1.
- Repeat until no updates happen in a full iteration:
    - Traverse from left to right:
        - If current rating is higher than left neighbor and candies are not more, increase candies.
    - Traverse from right to left:
        - If current rating is higher than right neighbor and candies are not more, increase candies.
- Sum all candies and return the result.

Example dry run (ratings = [1,0,5]):
Initial candies = [1,1,1]
Right to left fix → candies = [2,1,2]
No more violations → total = 5

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int candy(vector<int>& ratings) {
        int n = ratings.size();
        vector<int> candies(n, 1);
        bool updated = true;

        while (updated) {
            updated = false;

            for (int i = 1; i < n; i++) {
                if (ratings[i] > ratings[i - 1] && candies[i] <=
candies[i - 1]) {
                    candies[i] = candies[i - 1] + 1;
                    updated = true;
```

```
                }
            }

            for (int i = n - 2; i >= 0; i--) {
                if (ratings[i] > ratings[i + 1] && candies[i] <=
candies[i + 1]) {
                    candies[i] = candies[i + 1] + 1;
                    updated = true;
                }
            }
        }
        return accumulate(candies.begin(), candies.end(), 0);
//candies vector ke saare elements ka sum nikaalo,
//aur starting sum 0 se shuru karo.
    }
};

int main() {
    Solution obj;
    vector<int> ratings = {1,0,5};
    cout << obj.candy(ratings);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(N^2)$

Because multiple full passes may be required and each pass takes O(N).

Space Complexity: O(N)

An extra array is used to store candies.

Approach 2: Better Approach

Algorithm

Instead of repeated updates, we fix the conditions using exactly two passes.

Steps:

- Initialize a candies array with all values as 1.
- Traverse from left to right:
    - If current rating is higher than left neighbor, give more candies.
- Traverse from right to left:
    - If current rating is higher than right neighbor, update candies using max value.
- Sum all candies and return.

Example dry run (ratings = [1,0,5]):
Left to right → candies = [1,1,2]
Right to left → candies = [2,1,2]
Total = 5

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int candy(vector<int>& ratings) {
        int n = ratings.size();
        vector<int> candies(n, 1);

        for (int i = 1; i < n; i++) {
            if (ratings[i] > ratings[i - 1])
                candies[i] = candies[i - 1] + 1;
        }

        for (int i = n - 2; i >= 0; i--) {
            if (ratings[i] > ratings[i + 1])
                candies[i] = max(candies[i], candies[i + 1] + 1);
        }
        return accumulate(candies.begin(), candies.end(), 0);
    }
```

```
};

int main() {
    Solution obj;
    vector<int> ratings = {1,0,5};
    cout << obj.candy(ratings);
    return 0;
}
```

Complexity Analysis
Time Complexity: O(N)
Two linear passes and one sum pass.

Space Complexity: O(N)
Extra candies array is used.

Approach 3: Optimal Approach

Algorithm
We treat the ratings as increasing and decreasing slopes.
Each increasing slope and decreasing slope adds candies.
The peak is counted twice, so we subtract the smaller slope length.

Steps:

- Start with `candies = n` (1 candy for each child).
- Traverse the array:
    - Count increasing slope and add candies.
    - Count decreasing slope and add candies.
    - Subtract `min(peak, valley)` to remove double count.
- Return total candies.

Example dry run (ratings = [1,0,5]):

- Decreasing slope: 1 → add 1
- Increasing slope: 1 → add 1
- Total = 3 + 2 = 5

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int candy(vector<int>& ratings) {
        int n = ratings.size();
        int candies = n;
        int i = 1;

        while (i < n) {
            if (ratings[i] == ratings[i - 1]) {
                i++;
                continue;
            }

            int peak = 0;
            while (i < n && ratings[i] > ratings[i - 1]) {
                peak++;
                candies += peak;
                i++;
            }

            int valley = 0;
            while (i < n && ratings[i] < ratings[i - 1]) {
                valley++;
                candies += valley;
                i++;
            }

            candies -= min(peak, valley);
```

```
        }
        return candies;
    }
};

int main() {
    Solution sol;
    vector<int> ratings = {1,3,6,8,9,5,3};
    cout << sol.candy(ratings);
    return 0;
}
```

Complexity Analysis
Time Complexity: O(N)
Each index is processed at most twice.

Space Complexity: O(1)
Only constant variables are used.

Peak element **dono loops me count ho jaata hai**:

- ek baar increasing me
- ek baar decreasing me

But:

> Peak ko **max(peak, valley)** candies chahiye,
> sum nahi.

Isliye:

- jo extra add ho gaya → subtract kar dete hain

FULL DRY RUN

Input

```
ratings = {1,3,6,8,9,5,3}
n = 7
candies = 7 (base)
```

Increasing slope

1 → 3 → 6 → 8 → 9

step	peak	candies
3>1	1	+1 → 8
6>3	2	+2 → 10
8>6	3	+3 → 13
9>8	4	+4 → 17

peak = 4

Decreasing slope

9 → 5 → 3

step	valley	candies
5<9	1	+1 → 18
3<5	2	+2 → 20

valley = 2

Overcount correction

```
candies -= min(4,2) = 2
```

Final:

```
candies = 18
```

✅ Final Answer

18

12. Shortest Job First (or SJF) CPU Scheduling

You are given a list of job durations, where each value represents the time required to complete a job.
The CPU follows the Shortest Job First (SJF) scheduling policy, which means the job with the smallest duration is executed first.
Your task is to calculate the average waiting time of all jobs.

The waiting time of a job is the total time it waits before it starts execution.

Example:
jobs = [3, 1, 4, 2, 5]
After sorting → [1, 2, 3, 4, 5]
Waiting times → [0, 1, 3, 6, 10]
Average waiting time = (0 + 1 + 3 + 6 + 10) / 5 = 4

Explanation:

Socho bank ki line hai:

- Person A → kaam: 1 min
- Person B → kaam: 10 min
- Person C → kaam: 2 min

❌ Agar B pehle chala gaya:

- A aur C ko 10 min wait karna padega

✅ Agar A → C → B chale:

- Sabka wait minimum hoga

👉 Chhota kaam pehle = sabka wait kam

Yahi hai SJF.

Approach:

Algorithm
Shortest Job First works by always executing the job with the minimum duration first.

Steps:

- Sort the jobs array in ascending order.
- Initialize `totalTime = 0` to track execution time so far.
- Initialize `waitTime = 0` to store total waiting time of all jobs.
- Traverse the sorted jobs array:
    - Add `totalTime` to `waitTime` (waiting time for current job).
    - Add current job duration to `totalTime`.
- After processing all jobs, divide total waiting time by number of jobs to get average waiting time.

Example dry run (jobs = [3,1,4,2,5]):
Sorted jobs = [1,2,3,4,5]
Job 1 → wait = 0, totalTime = 1

Job 2 → wait = 1, totalTime = 3
Job 3 → wait = 3, totalTime = 6
Job 4 → wait = 6, totalTime = 10
Job 5 → wait = 10, totalTime = 15
Average = (0+1+3+6+10)/5 = 4

Code

```
#include <bits/stdc++.h>
using namespace std;

class ShortestJobFirst {
public:
    float calculateAverageWaitTime(vector<int>& jobs) {
        sort(jobs.begin(), jobs.end());

        float waitTime = 0;
        int totalTime = 0;
        int n = jobs.size();

        for (int i = 0; i < n; i++) {
            waitTime += totalTime;
            totalTime += jobs[i];
        }
        return waitTime / n;
    }
};

int main() {
    vector<int> jobs = {4,3,7,1,2};
    ShortestJobFirst sjf;
    cout << sjf.calculateAverageWaitTime(jobs);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N log N)

Sorting the jobs array takes $O(N \log N)$ time, and calculating waiting time takes $O(N)$.

Space Complexity: O(1)
No extra space is used apart from variables; sorting is done in-place.

13. Program for Least Recently Used (LRU) Page Replacement Algorithm

We need to design an LRU (Least Recently Used) Cache with a fixed capacity.
The cache supports two operations:

- `get(key)`:
  Return the value of the key if it exists in the cache, otherwise return `-1`.
  Accessing a key makes it the most recently used.

- `put(key, value)`:
  Insert or update the key with the given value.
  If the cache exceeds its capacity, remove the least recently used key.

Both operations must work in **O(1)** average time.

Example:
Capacity = 2
put(1,1) → cache = {1}
put(2,2) → cache = {1,2}
get(1) → returns 1, cache = {2,1}
put(3,3) → evict 2, cache = {1,3}

Approach 1

Algorithm

To achieve O(1) time for both operations, we combine two data structures:

- **HashMap**:
  Stores key → node mapping for fast lookup.

- **Doubly Linked List**:
  Maintains usage order.
    - Most recently used node is near the head.
    - Least recently used node is near the tail.

Steps:

- Use dummy head and tail nodes to simplify insert and delete operations.
- On `get(key)`:
    - If key exists, move the node to the front (most recently used) and return its value.
    - If not found, return -1.
- On `put(key, value)`:
    - If key already exists, remove the old node.
    - If cache is full, remove the node just before the tail (least recently used).
    - Insert the new node at the front.
- Always keep HashMap updated with the latest node positions.

Example dry run (capacity = 2):

- put(1,1) → [1]
- put(2,2) → [2,1]
- get(1) → move 1 to front → [1,2]
- put(3,3) → remove 2 → [3,1]

Code

```
#include <bits/stdc++.h>
```

```
using namespace std;

class LRUCache {
public:
    class Node {
    public:
        int key, val;
        Node* next;
        Node* prev;
        Node(int k, int v) {
            key = k;
            val = v;
            next = prev = NULL;
        }
    };

    Node* head = new Node(-1, -1);
    Node* tail = new Node(-1, -1);
    int cap;
    unordered_map<int, Node*> mp;

    LRUCache(int capacity) {
        cap = capacity;
        head->next = tail;
        tail->prev = head;
    }

    void addNode(Node* node) {
        Node* temp = head->next;
        node->next = temp;
        node->prev = head;
        head->next = node;
        temp->prev = node;
    }

    void deleteNode(Node* node) {
        Node* prevNode = node->prev;
        Node* nextNode = node->next;
```

```
        prevNode->next = nextNode;
        nextNode->prev = prevNode;
    }

    int get(int key) {
        if (mp.find(key) != mp.end()) {
            Node* node = mp[key];
            int res = node->val;
            mp.erase(key);
            deleteNode(node);
            addNode(node);
            mp[key] = head->next;
            return res;
        }
        return -1;
    }

    void put(int key, int value) {
        if (mp.find(key) != mp.end()) {
            Node* node = mp[key];
            mp.erase(key);
            deleteNode(node);
        }

        if (mp.size() == cap) {
            mp.erase(tail->prev->key);
            deleteNode(tail->prev);
        }

        addNode(new Node(key, value));
        mp[key] = head->next;
    }
};

int main() {
    LRUCache cache(2);
    cache.put(1,1);
    cache.put(2,2);
```

```
    cout << cache.get(1) << endl;
    cache.put(3,3);
    cout << cache.get(2) << endl;
    cache.put(4,4);
    cout << cache.get(1) << endl;
    cout << cache.get(3) << endl;
    cout << cache.get(4) << endl;
    return 0;
}
```

Complexity Analysis
Time Complexity:

- `get()` → O(1) average time due to HashMap lookup and constant list operations.
- `put()` → O(1) average time for insert, delete, and eviction.

Space Complexity: O(capacity)
We store at most `capacity` nodes in the doubly linked list and HashMap.

14. Insert Interval

You are given a list of non-overlapping intervals sorted by their start time.
You are also given a new interval.
Your task is to insert the new interval into the list and merge all overlapping intervals, then return the final list of intervals.

Example:
intervals = [[1,3],[4,5],[6,7],[8,10]]
newInterval = [5,6]

After insertion and merging, the result becomes:
[[1,3],[4,7],[8,10]]

Approach 1: Naive Approach (Insertion and Merging)

Algorithm

The idea is simple. We first insert the new interval into the list and then merge all overlapping intervals using the same logic as the “Merge Overlapping Intervals” problem.

Steps:

- Add the new interval to the intervals list.
- Sort the intervals based on starting time.
- Initialize a result list and add the first interval.
- Traverse the remaining intervals one by one.
- If the current interval overlaps with the last interval in result, merge them.
- Otherwise, add the current interval to result.
- Return the merged list.

Example dry run:
intervals = [[1,3],[4,5],[6,7],[8,10]]
newInterval = [5,6]
After insertion → [[1,3],[4,5],[6,7],[8,10],[5,6]]
After sorting → [[1,3],[4,5],[5,6],[6,7],[8,10]]
Merging overlapping intervals → [[1,3],[4,7],[8,10]]

Code

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

vector<vector<int>> mergeOverlap(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end());
    vector<vector<int>> res;
    res.push_back(intervals[0]);

    for (int i = 1; i < intervals.size(); i++) {
```

```
        vector<int>& last = res.back();
        vector<int>& curr = intervals[i];

        if (curr[0] <= last[1])
            last[1] = max(last[1], curr[1]);
        else
            res.push_back(curr);
    }
    return res;
}

vector<vector<int>> insertInterval(vector<vector<int>>& intervals,
                                   vector<int>& newInterval) {
    intervals.push_back(newInterval);
    return mergeOverlap(intervals);
}

int main() {
    vector<vector<int>> intervals = {{1,3},{4,5},{6,7},{8,10}};
    vector<int> newInterval = {5,6};

    vector<vector<int>> res = insertInterval(intervals, newInterval);
    for (auto interval : res) {
        cout << interval[0] << " " << interval[1] << endl;
    }
    return 0;
}
```

Complexity Analysis

Time Complexity: O(n log n)

Sorting the intervals takes O(n log n), and merging takes O(n).

Space Complexity: O(1) (excluding output)

No extra data structure is used apart from the result array.

Approach 2: Expected Approach (Contiguous Interval Merging)

Algorithm

Since the intervals are already sorted, only a contiguous group of intervals can overlap with the new interval. We directly find and merge only those intervals.

Steps:

- Add all intervals that end before the new interval starts.
- Merge all intervals that overlap with the new interval by expanding its range.
- Add the merged interval to the result.
- Add all remaining intervals after the merged interval.
- Return the result list.

Example dry run:
intervals = [[1,3],[4,5],[6,7],[8,10]]
newInterval = [5,6]

- Add intervals before overlap → [[1,3]]
- Merge overlapping intervals → newInterval becomes [4,7]
- Add merged interval → [[1,3],[4,7]]
- Add remaining intervals → [[1,3],[4,7],[8,10]]

Code

```
#include <iostream>
#include <vector>
using namespace std;

vector<vector<int>> insertInterval(vector<vector<int>>& intervals,
                                   vector<int>& newInterval) {
    vector<vector<int>> res;
    int i = 0;
    int n = intervals.size();
```

```
    while (i < n && intervals[i][1] < newInterval[0]) {
        res.push_back(intervals[i]);
        i++;
    }

    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = min(newInterval[0], intervals[i][0]);
        newInterval[1] = max(newInterval[1], intervals[i][1]);
        i++;
    }
    res.push_back(newInterval);

    while (i < n) {
        res.push_back(intervals[i]);
        i++;
    }
    return res;
}

int main() {
    vector<vector<int>> intervals = {{1,3},{4,5},{6,7},{8,10}};
    vector<int> newInterval = {5,6};

    vector<vector<int>> res = insertInterval(intervals, newInterval);
    for (auto interval : res) {
        cout << interval[0] << " " << interval[1] << endl;
    }
    return 0;
}
```

Complexity Analysis

Time Complexity: O(n)

Each interval is processed only once.

Space Complexity: O(n)

A new result array is used to store the merged intervals.

15. Merge Overlapping Sub-intervals

You are given an array of intervals where each interval is represented as `[start, end]`.
If two intervals overlap, they should be merged into a single interval.
The final result should contain only non-overlapping intervals that cover all given intervals.

Example:
intervals = [[1,3],[2,6],[8,10],[15,18]]
Merged result = [[1,6],[8,10],[15,18]]

Approach 1: Brute-Force Approach

Algorithm
The idea is to merge intervals by checking overlaps manually after sorting.

Steps:

- Sort all intervals based on their starting time.
- Traverse the intervals one by one.
- Take the current interval as a new merged interval.
- Check the next intervals to see if they overlap with the current one.
- If an interval overlaps, extend the end of the current interval.
- Keep merging until no more overlaps are found.
- Store the merged interval and move to the next non-overlapping interval.

Example dry run:
intervals = [[1,3],[2,6],[8,10],[15,18]]
Sorted → same order

- Start with [1,3], overlaps with [2,6] → merge to [1,6]
- [8,10] does not overlap → keep as is
- [15,18] does not overlap → keep as is

Final answer = [[1,6],[8,10],[15,18]]

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        vector<vector<int>> ans;
        int n = intervals.size();

        for (int i = 0; i < n; ) {
            int start = intervals[i][0];
            int end = intervals[i][1];
            int j = i + 1;

            while (j < n && intervals[j][0] <= end) {
                end = max(end, intervals[j][1]);
                j++;
            }
            ans.push_back({start, end});
            i = j;
        }
        return ans;
    }
};

int main() {
    Solution sol;
    vector<vector<int>> intervals = {{1,3},{2,6},{8,10},{15,18}};
    vector<vector<int>> result = sol.merge(intervals);

    for (auto interval : result) {
        cout << "[" << interval[0] << "," << interval[1] << "] ";
    }
    return 0;
```

}

Complexity Analysis
Time Complexity: O(N²)
For each interval, we may scan many future intervals to check overlap.

Space Complexity: O(N)
Extra space is used to store the merged intervals.

Approach 2: Optimal Approach

Algorithm
Instead of checking all future intervals, we only compare each interval with the last merged one.

Steps:

- Sort intervals based on starting time.
- Create an empty result list.
- Traverse each interval:
    - If result is empty or current interval does not overlap with the last one, add it.
    - If it overlaps, merge by updating the end of the last interval.
- Return the result list.

Example dry run:
intervals = [[1,3],[2,6],[8,10],[15,18]]

- Add [1,3]
- [2,6] overlaps → merge to [1,6]
- [8,10] does not overlap → add
- [15,18] does not overlap → add

Final answer = [[1,6],[8,10],[15,18]]

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        vector<vector<int>> merged;

        for (auto interval : intervals) {
            if (merged.empty() || merged.back()[1] < interval[0]) {
                merged.push_back(interval);
            } else {
                merged.back()[1] = max(merged.back()[1], interval[1]);
            }
        }
        return merged;
    }
};

int main() {
    Solution sol;
    vector<vector<int>> intervals = {{1,3},{2,6},{8,10},{15,18}};
    vector<vector<int>> result = sol.merge(intervals);

    for (auto interval : result) {
        cout << "[" << interval[0] << "," << interval[1] << "] ";
    }
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N log N)
Sorting takes O(N log N) and merging takes O(N).

Space Complexity: O(N)
Extra space is used to store the merged intervals.

16. Non-overlapping Intervals

You are given N intervals, where each interval is represented as `[start, end]`.
Two intervals are overlapping if one starts before the other ends.
Your task is to return the **minimum number of intervals to remove** so that the remaining intervals are **non-overlapping**.

Example:
Intervals = [[1,2],[2,3],[3,4],[1,3]]
If we remove `[1,3]`, the remaining intervals do not overlap.
So, the answer is 1.

Approach 1: Brute Force Approach

Algorithm
The brute force idea is to try all possible subsets of intervals and check which subsets are non-overlapping. Among all valid (non-overlapping) subsets, we find the one with the maximum number of intervals kept. The number of intervals to remove is the total intervals minus this maximum valid subset size.

Steps:

- Generate all possible subsets of the intervals using bitmasking.
- For each subset:
    - Sort the selected intervals by start time.
    - Check if any two consecutive intervals overlap.
- If the subset is valid (non-overlapping), update the maximum size found.
- Return `total intervals - maximum valid subset size`.

Example dry run (Intervals = [[1,2],[2,3],[3,4],[1,3]]):

- One valid subset is [[1,2],[2,3],[3,4]] with size 3.
- Total intervals = 4

- Minimum removals = 4 − 3 = 1

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        int n = intervals.size();
        int maxValid = 0;

        for (int mask = 0; mask < (1 << n); mask++) {
            vector<vector<int>> subset;

            for (int i = 0; i < n; i++) {
                if (mask & (1 << i))
                    subset.push_back(intervals[i]);
            }

            sort(subset.begin(), subset.end());

            bool valid = true;
            for (int i = 1; i < subset.size(); i++) {
                if (subset[i][0] < subset[i - 1][1]) {
                    valid = false;
                    break;
                }
            }

            if (valid)
                maxValid = max(maxValid, (int)subset.size());
        }

        return n - maxValid;
    }
};
```

```
int main() {
    Solution sol;
    vector<vector<int>> intervals = {{1,2},{2,3},{3,4},{1,3}};
    cout << sol.eraseOverlapIntervals(intervals);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(2^n \times N \log N)$
There are 2^n subsets. For each subset, we sort and check overlaps.

Space Complexity: O(N)
Extra space is used to store a subset of intervals.

Approach 2: Optimal Approach (Greedy)

Algorithm

To keep the maximum number of non-overlapping intervals, we greedily select intervals that finish earliest. This ensures that we leave maximum room for future intervals.

Steps:

- Sort all intervals by their ending time.
- Initialize `prevEnd` as the end of the first interval.
- Initialize a counter for removals.
- Traverse the remaining intervals:
    - If the current interval starts before `prevEnd`, it overlaps, so increment removal count.
    - Otherwise, update `prevEnd` to the current interval's end.
- Return the removal count.

Example dry run (Intervals = [[1,2],[2,3],[3,4],[1,3]]):
After sorting by end → [[1,2],[1,3],[2,3],[3,4]]

- Keep [1,2], prevEnd = 2
- [1,3] overlaps → remove
- [2,3] does not overlap → keep, prevEnd = 3
- [3,4] does not overlap → keep
  Removals = 1

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end(),
             [](vector<int>& a, vector<int>& b) {
                 return a[1] < b[1];
             });

        int count = 0;
        int prevEnd = intervals[0][1];

        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] < prevEnd) {
                count++;
            } else {
                prevEnd = intervals[i][1];
            }
        }
        return count;
    }
};

int main() {
    Solution sol;
    vector<vector<int>> intervals = {{1,3},{2,4},{3,5},{1,2}};
```

```
    cout << sol.eraseOverlapIntervals(intervals);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N log N)

Sorting the intervals takes `O(N log N)` and traversal takes `O(N)`.

Space Complexity: O(1)

Only constant extra variables are used.